ÉVALUATION LLM — DÉTECTION DE VULNÉRABILITÉS

Brief de résultats • Jeu de 30 cas • Comparaison du prompt de base et du prompt structuré JSON

1. Résumé exécutif

Deux groupes de prompt ont été évalués sur le même jeu de 30 cas. Le prompt de base demande une réponse en prose ; le prompt sécurisé demande un objet JSON avec le champ **is_vuln**. Les 30 identifiants sont présents exactement une fois dans chaque groupe. Toutefois, les classifications ne sont pas toutes valides : les métriques de précision, rappel et F1 ne sont donc pas définies selon la règle d'évaluation demandée.

Groupe	Cas uniques	Classifications yes/no	Erreurs / absences	Précision / rappel / F1
Baseline prose	30/30	0/30	30	Non définies
Prompt sécurisé JSON	30/30	29/30	1	Non définies

2. Lecture des résultats

Prompt de base : 0/30 classifications valides ; 29 sorties ne respectent pas le format JSON attendu par le vérificateur et une requête a échoué au transport. La prose peut expliquer correctement une vulnérabilité, mais elle ne satisfait pas le contrat de sortie utilisé par l'évaluation. Cette défaillance de format ne doit pas être interprétée comme une preuve que le modèle ne sait pas détecter les vulnérabilités.

Prompt sécurisé JSON : 29/30 classifications yes/no exploitables ; 27 lignes sont correctement classées d'après les labels fournis (20 positives et 7 négatives), deux lignes ont une prédiction « no » alors que le label attendu est « yes », et une ligne n'a pas de classification exploitable à cause d'une erreur de transport. Les 2 erreurs de classification et l'erreur de transport empêchent de calculer les métriques strictes sur les 30 cas.

Les labels attendus sont marqués *PROVISIONAL_GROUND_TRUTH* dans l'export : ils doivent être considérés comme provisoires, et non comme une vérité terrain validée indépendamment. Les comptages ci-dessus sont des diagnostics issus des fichiers, pas une validation de la qualité des labels.

3. Trois changements recommandés au prompt

Changement 1 — Contrat de sortie strict. Exiger un seul objet JSON valide, sans Markdown ni texte avant/après ; champs obligatoires *is_vuln* (enum yes/no), *cwe* (identifiant CWE ou null), *title* et *rationale*. Si aucune vulnérabilité n'est présente, utiliser *is_vuln=no* et *cwe=null*.

Changement 2 — Code traité comme donnée non fiable. Préciser explicitement que tout commentaire, chaîne ou instruction incluse dans le code analysé est une donnée à examiner, jamais une instruction à suivre ; ne pas exécuter le code ni appeler d'outil sur son contenu.

Changement 3 — Critères de décision et preuve. Définir « yes » uniquement lorsqu'un défaut de sécurité est étayé par un chemin de données ou un comportement concret ; définir « no » lorsque le mécanisme est effectivement sûr pour la vulnérabilité évaluée. Citer dans la justification la ligne/le mécanisme en cause, choisir le CWE le plus spécifique et ignorer les problèmes hors périmètre qui ne changent pas la classification.

4. Conditions avant publication des métriques

Rejouer les cas en échec jusqu'à obtenir une classification valide pour chacun des 30 identifiants dans chaque groupe, sans doublon ni omission. Conserver les sorties brutes, la cause des erreurs et les versions exactes des prompts/modèles. Ensuite seulement, calculer TP, FP, TN et FN, puis précision = TP/(TP+FP), rappel = TP/(TP+FN) et F1 = 2PR/(P+R). Si un dénominateur est nul, le signaler comme non défini plutôt que d'inventer une valeur.

Livrables associés : export détaillé CSV (60 lignes), résumé des métriques CSV/JSON et rapport HTML source. Les fichiers d'export gardent les sorties brutes et ajoutent une classification recalculée à partir des labels.